Computer Science & Engineering Department

Arhitectura Sistemelor de Calcul

Curs 1
cs.pub.ro
cs.curs.pub.ro

University Politehnica of Bucharest

Titulari de Curs

- Prof. Emil-Ioan Sluşanschi – Seriile CB & CD
  emil.slusanschi@cs.pub.ro



- Șl. Voichița Iancu – Seria CA & CC
  voichita.iancu@cs.pub.ro

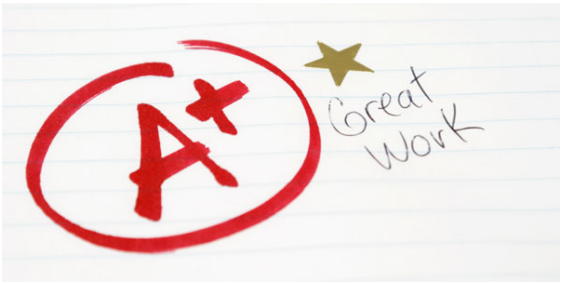


Cuprins

- Notare
- Sumarul Cursului
- Bibliografie
- Educatia Inginereasca
- De ce Arhitectura Sistemelor de Calcul?
- Idei Importante in Arhitectura Sistemelor de Calcul

Notarea la ASC

- Laborator: 5/5.5
  - Trei teme: 4.5:
    - Python: 1.5
    - GP-GPU: 1.5
    - Optimizare C: 1.5
  - Activitate la laborator: 1
  - Bonusuri:
    - Max 20% @teme
    - Participări concursuri
- Examen: 5 (teorie)
  - Min > 2.5 lab
  - Oral || Scris



Colaborarea in **Negru** si Alb

- **Colaborarea Buna**
  - Discutii, brainstorming, dezbateri on/off-line
  - Participarea activa la prezentarile temelor in laboratoarele dedicate
- **Colaborarea Rea**
  - Lucrul in comun/revizie pe cod
  - Reluare/oferire /partajare/copiere de (segmente de) cod
- Ar trebui sa fie de la sine inteles, dar…
  - Nu cumparati teme!
  - Nu rugati pe cineva din afara cursului sa va ajute
  - Nu cautati pe internet / GitHub / ChatGPT solutii pentru teme
  - Nu puneti teme rezolvate pe site-uri nesecurizate sau pe GitHub / GitLab
  - Doar din propriile teme veti invata cu adevarat



Sumarul Cursului

Un Scurt Sumar al Cursului

- Intro & Recapitulare
- Nivelul structural de descriere a sitemelor numerice - PMS
- Ierarhia de memorii, localitatea datelor, cache
- Arhitecturi omogene/eterogene: CPU / GP-GPU
- Comutatoare ierarhice & neierarhice
- Structuri de calcul cu prelucrare paralela – taxonomii
- Structuri SIMD
- Intercalarea perfecta si conectarea inversa

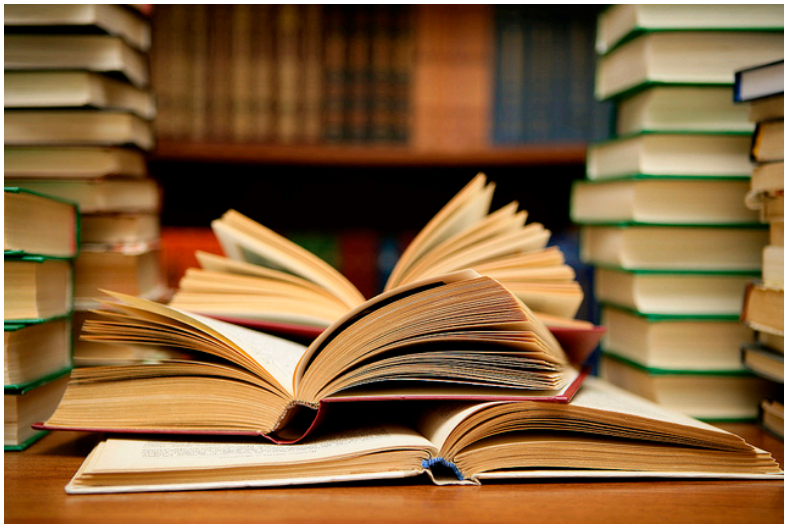


Un Scurt Sumar al Cursului (2)

- Arhitecturi MIMD – Caracteristici
- Arhitectura Cm* - comunicare intra- si inter-cluster
- Interconectarea in sisteme cu resurse multiple
- Retele de comutare (Ierarhice & Delta)
- Debugging, profiling, analiza de performanta
- Top 500, Benchmarking & the current Top 10

  Sumarul cursului se poate modifica pe parcursul semestrului





Bibliografie

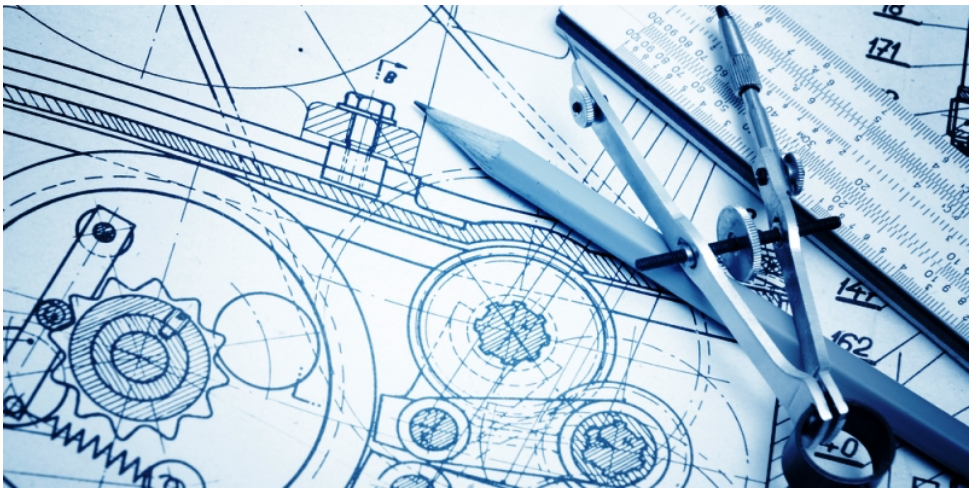
Bibliografie

- Structura si Arhitectura Sistemelor Numerice; T. Moisa, N. Tapus, C. Morarescu 1999
- Introduction to Parallel Computing: Design & Analysis of Algorithms; V. Kumar, A. Grama, A. Gupta, G. Karypis; Addison Wesley; 2nd Edition 2003
- The Sourcebook of Parallel Computing; J. Dongarra, I. Foster, W. Grapp, K. Kennedy; Morgan Kaufmann 2002
- Computer Organization & Design – The Hardware Software Interface; D.A. Patterson, J. Hennesy; Morgan Kaufmann; 5th Edition 2014
- Computer Architecture: A Quantitative Approach; J. Hennesy, D.A. Patterson; Morgan Kaufmann; 6th Edition 2017
- Introduction to Parallel Processing; Algorithms and Architectures; Behrooz Parhami; Kluwer 2002
- Techniques for Optimizing Applications: High Performance Computing; Rajat P. Garg, Ilya Sharapov; Sun 2001.
- Inside the Machine; Jon Stokes; No Starch Press 2006.
- www.top500.org



Bibliografie vs. Cursuri

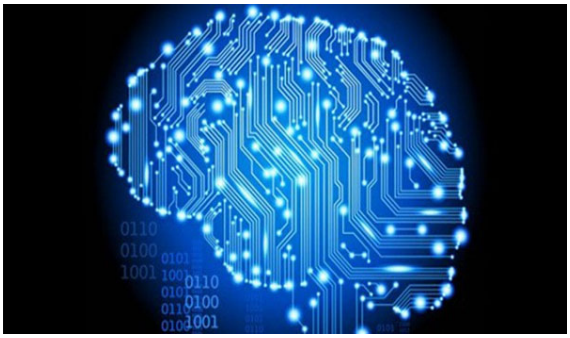
- Structura si Arhitectura Sistemelor Numerice
  - Cursurile 2, 3, 6, 7, 8, 9, 10, 11, 12
- Introduction to Parallel Computing: Design & Analysis of Algorithms
  - Cursurile 6, 8, 11
- The Sourcebook of Parallel Computing, www.top500.org, netlib.org, icl.cs.utk.edu/hpcc/
  - Cursurile 13, 14
- Computer Organization & Design – The Hardware Software Interface
  - Cursurile 2-14
- Computer Architecture: A Quantitative Approach
  - Cursurile 3, 4, 7, 9, 10, 11
- Introduction to Parallel Processing; Algorithms and Architectures
  - Cursurile 6-11
- Techniques for Optimizing Applications
  - Cursul 4
- Inside the Machine; Jon Stokes
  - Cursurile 1, 4, 5, 7, 8



Educatie Inginereasca

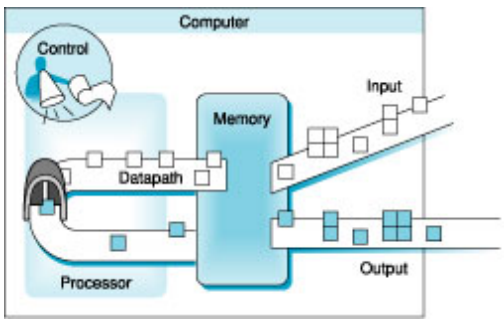
Ce este un Computer?

- Un instrument pentru a rezolva probleme
  - Prelucrarea datelor/numerelor
  - Prelucrarea algoritmilor
- Un procesor ce controleaza un sistem (avion, fabrica, monitor de inima, trafic, robot, etc.)
  - Senzori / Intrari
  - Actuator / Iesiri
  - Functii
  - Stari



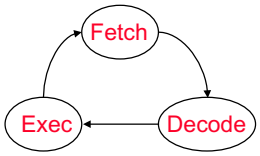
@Yale Patt - University of Texas at Austin

Ce este un Computer? (2)



- Components:
  - Processor (Datapath & Control)
  - Input (mouse, keyboard)
  - Output (display, printer)
  - Memory (cache, main memory, disks, CD/DVD)
  - Network
- Our primary focus: the Processor (**Datapath** & **Control**)
  - Implemented using billions of transistors
  - Impossible to understand by looking at each transistor
  - We need abstraction!

Processor Organisation



- **Control** needs to have circuitry to
  - Decide which is the next instruction and input it from memory
  - Decode the instruction
  - Issue signals that control the way information flows between datapath components
  - Control what operations the datapath's functional units perform
- **Datapath** needs to have circuitry to
  - Execute instructions - functional units (e.g. adder, multiplier, etc.) and storage locations (e.g. register file)
  - Interconnect the functional units so that the instructions can be executed as required
  - Load data from and store data to memory



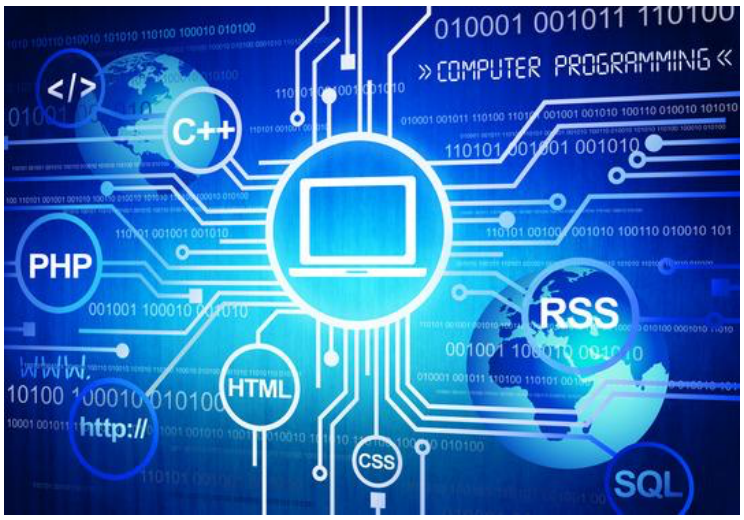
Typical Design Goals for Processors

- Maximize performance (runtime, throughput, etc.)
- Minimize memory space (embedded systems)
- Minimize power consumption (mobile systems)
- Reduce design time (time-to-market)
- Minimize cost ($)



The Four Design Principles (CN)

- Simplicity favours regularity
- Smaller is faster
- Make the common case fast
- Good design demands good compromises



De ce ASC?

Why Computer Organization?

- Embarrassing if you are a BS in CSE and can't make sense of the following terms: DRAM, pipelining, cache hierarchies, I/O, virtual memory, …
- Embarrassing if you are a BS in CSE and can't decide which processor to buy wrt different metrics: 2.2 GHz Xeon or 3.6 GHz A8
  - E.g. Performance vs. Power
- Obvious first step for chip designers, compiler & OS writers
- **Will knowledge of the hardware help you write better programs?**



Must a Programmer **care** about Hardware?

- Must know how to reason about program performance and energy
- Memory management: if we understand how/where data is placed, we can help ensure that **relevant data** is **nearby**
- Thread management: if we understand how threads **interact**, we can write **smarter** multi-threaded programs
- Why do we care about multi-threaded programs?
  - **Because most HW is now multi/many-core**



Ce vom discuta la ASC?

- Discussions about the hardware-software interface
  - What do you need to know to achieve best possible performance
- Introduction to the design of major components of a computer system, how they function together in an executing program
- But what do we mean by a computer?
  - Different types: mobile, embedded, laptop, desktop, server,…
  - Different uses: AI, auto, graphics, finance, genomics,…
- Best way to learn:
  - Focus on a specific instance and learn how it works
  - While learning general principles and historical perspectives

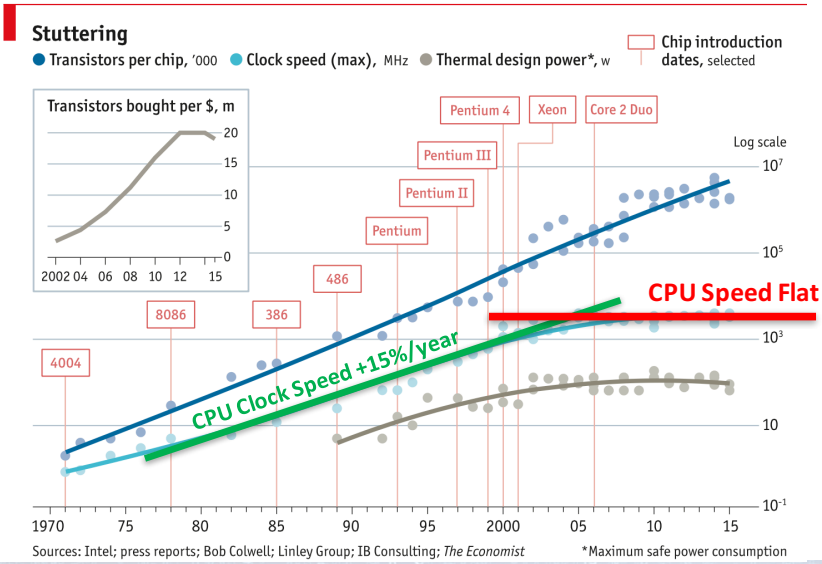


De ce sa discutam de HW/SW?

- You want to call yourself a “computer scientist/engineer”
- You want to build HW/SW people use
  - So you need to deliver **performance** at **low cost**
- You need to make a purchasing decision or offer “expert” advice
- Both hardware and software affect performance
  - The **algorithm** determines number of **source-level statements**
  - The **language/compiler/architecture** determines the number of machine **instructions**
  - The **processor/memory** determines **how fast** machine-level instructions are **executed**



Why is Architecture Exciting Today?





ACM Communications, February 2019

Important Trends

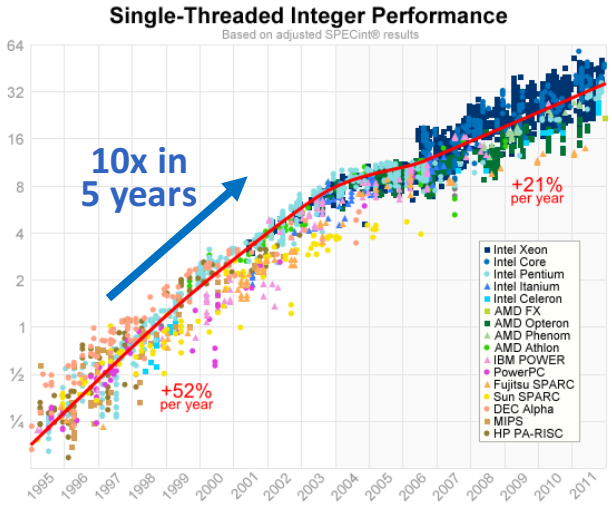
- Running out of ideas to improve single thread performance
- Power wall makes it harder to add complex features
- Power wall makes it harder to increase frequency
- Historical contributions to performance:
  - Better processes (faster devices) ~**20%** (eventually disappearing)
  - Better circuits/pipelines ~**15%** (trending lower)
  - Better organization/architecture ~**15%**

Practical Example

- +**300x** speedup for matrix vector multiplication
  - Data level parallelism: **3.8x**
  - Loop unrolling and out-of-order execution: **2.3x**
  - Cache blocking: **2.5x**
  - Thread level parallelism: **14x**
- Further, one can use accelerators to get an additional **100x**

Predicting the Future

- Predictable performance improvements
- Predictable market
  - Everybody buys a new CPU every X years
- Predictable expenses
  - Cost of new integrated circuit fab
- CEOs and Wall Street love this





Opportunities



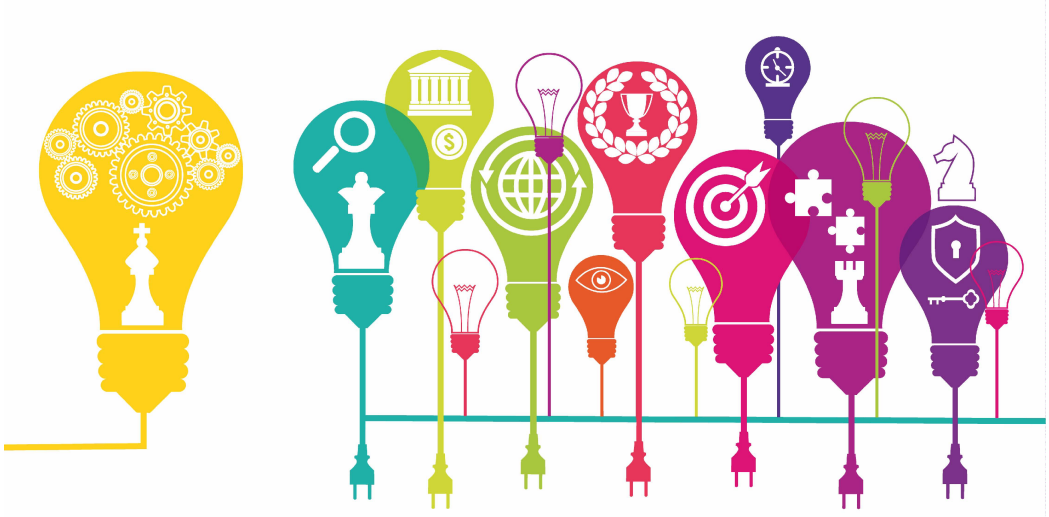
- Computer architecture today
  - General purpose: optimized for speed
  - Few players: AMD, ARM, Intel, NVidia…
- Future directions?
  - Applications have different requirements
    - Battery life / energy
    - Security & privacy
- Best solutions depend on application
  - Many players: HW / SW
  - **Innovation matters more than Moore's Law**

What Does This Mean to a Programmer?

- Today, one can expect **at most a 20% annual improvement** / even lower if the program is not multi-threaded
  - A program needs **many threads**
  - The threads need **efficient synchronization** and **communication** (Tema 1)
  - **Accelerators** should be used whenever possible (Tema 2)
  - **Data placement** in the **memory hierarchy** is important (Tema 3)

Challenges for Hardware Designers

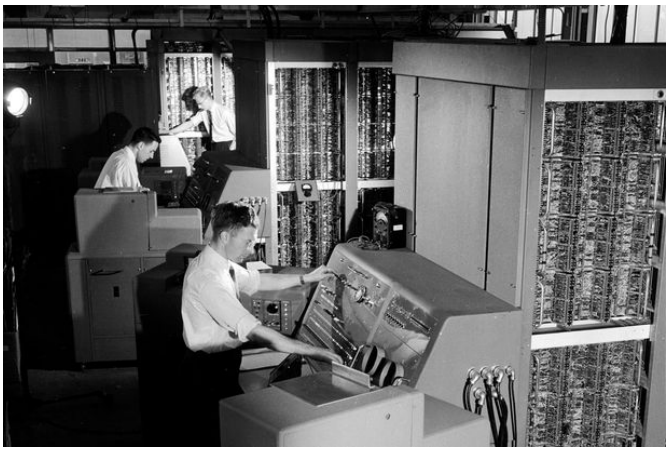
- Find efficient ways to
  - **Improve data sharing & synch techniques** (Tema 1)
  - **Boost programmer productivity** (Temele 1-3)
  - **Manage the memory system** (Temele 2-3)
  - **Build accelerators for important kernels** (Tema 2)
  - **Boost single-thread performance** (Tema 3)
  - Reduce system energy per instruction

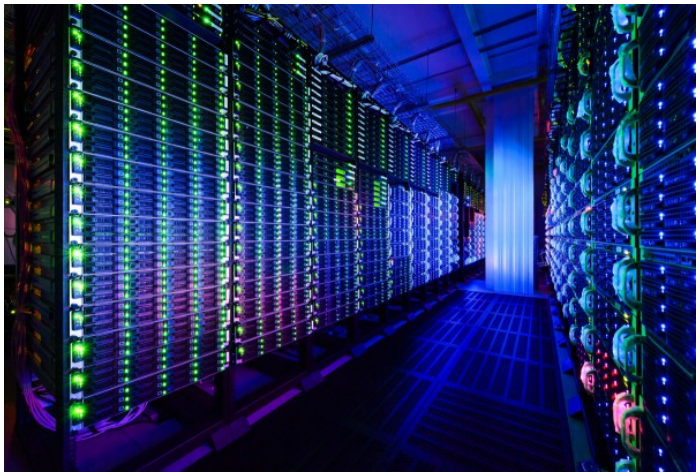


Idei Importante in Arhitectura Sistemelor de Calcul

Sisteme de Calcul

Atunci…





Sisteme de Calcul

Acum.

Arhitectura Sistemelor de Calcul

Software

- Parallel Requests
  Assigned to computer
  e.g., Search "cats"
- Parallel Threads
  Assigned to core
  e.g., Lookup, Ads
- Parallel Instructions
  >1 instruction @ one time
  e.g., 5 pipelined instructions
- Parallel Data
  >1 data item @ one time
  e.g., Add of 4 pairs of words
- Hardware descriptions
  All gates functioning in parallel at same time

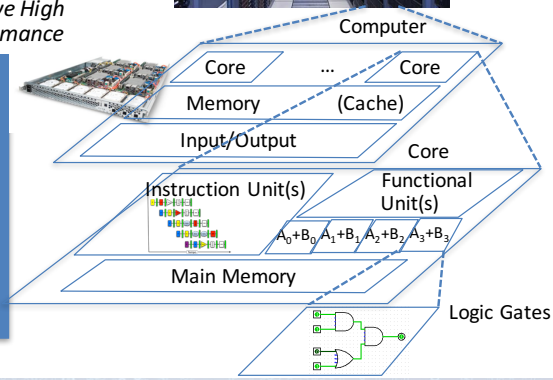
Harness Parallelism & Achieve High Performance

Hardware



Warehouse -Scale Computer

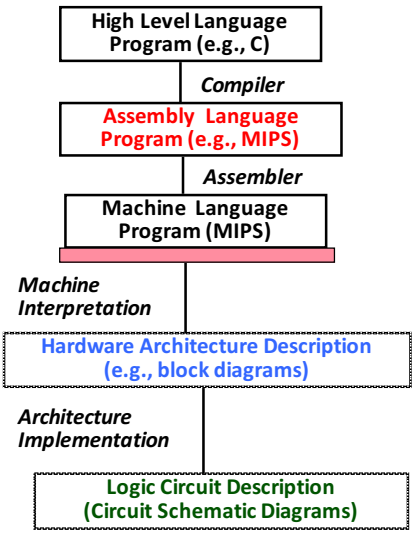
Smart Phone



Cinci Idei Importante in Arhitectura Sistemelor de Calcul

1. Abstraction - Layers of Representation / Interpretation
2. Moore's Law - Designing through trends
3. Principle of Locality - Memory Hierarchy
4. Parallelism
5. Dependability via Redundancy

Ideea 1: Abstractizarea

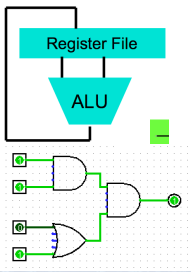



```
temp = v[k];
v[k] = v[k+1];
v[k+1] = temp;
```

```
lw   $t0, 0($2)
lw   $t1, 4($2)
sw   $t1, 0($2)
sw   $t0, 4($2)
```

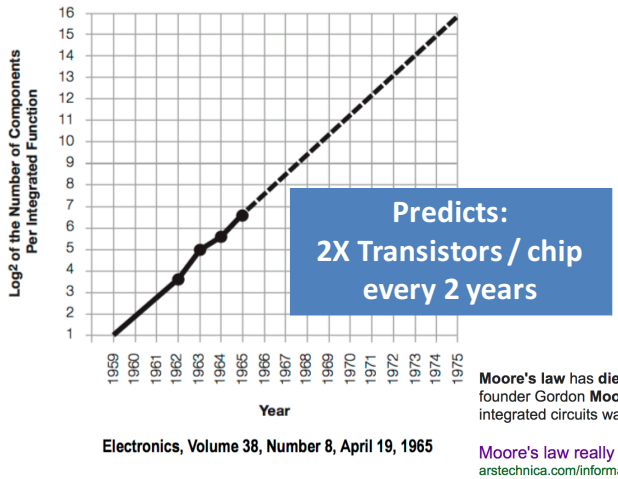
Anything can be represented as a *number*, i.e., data or instructions

```
0000 1001 1100 0110 1010 1111 0101 1000
1010 1111 0101 1000 0000 1001 1100 0110
1100 0110 1010 1111 0101 [illegible]  1
0101 1000 0000 1001 1100 [illegible]  1
```

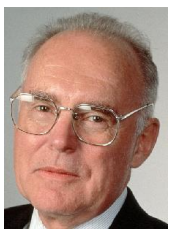




Ideea 2: Legea lui Moore



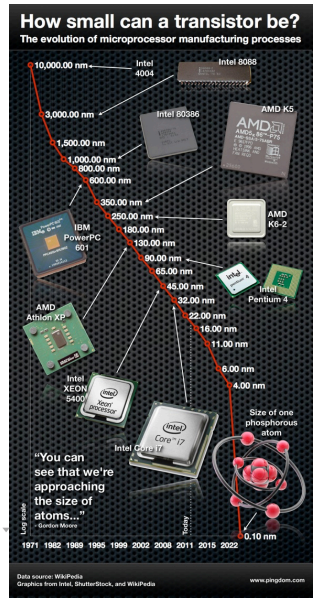
Electronics, Volume 38, Number 8, April 19, 1965



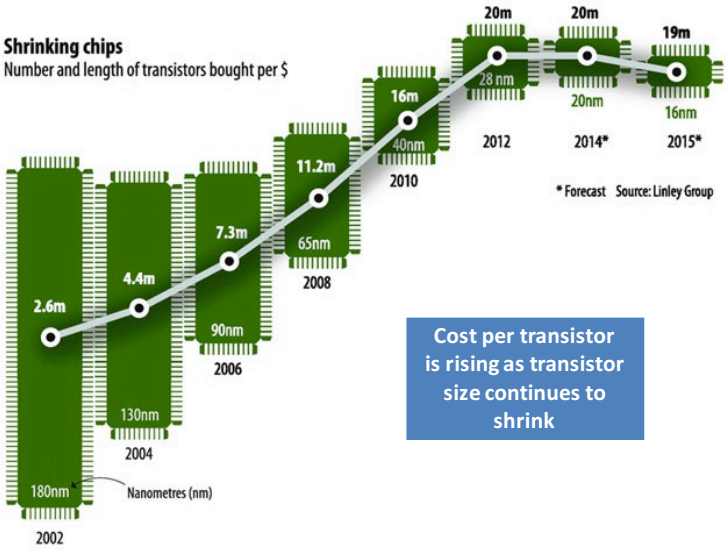
Gordon Moore
Intel Cofounder
B.S. Cal 1950!

Moore's law has **died** at the age of 51 after an extended illness. In 1965, Intel co-founder Gordon **Moore** made an observation that the number of components in integrated circuits was doubling every 12 months or so. Feb 11, 2016

Moore's law really is dead this time | Ars Technica
arstechnica.com/information.../2016/.../**moores-law**-really-is-dead-this-time... Ars Technica



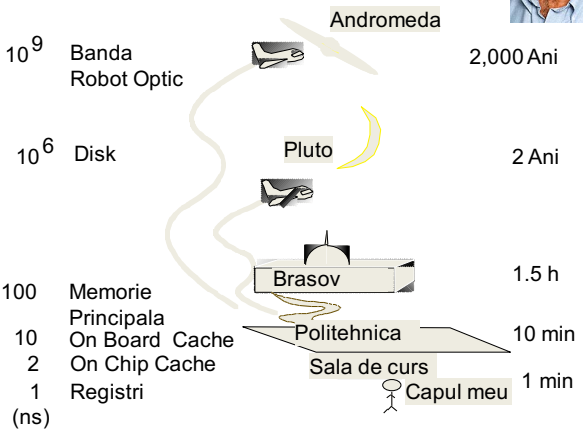
Numarul de tranzistori si latenta



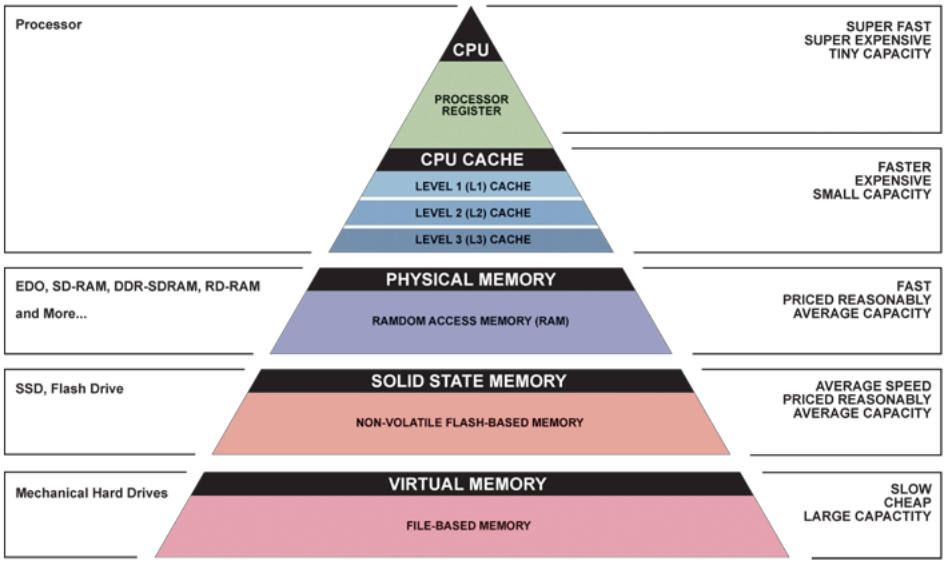
Cost per transistor is rising as transistor size continues to shrink



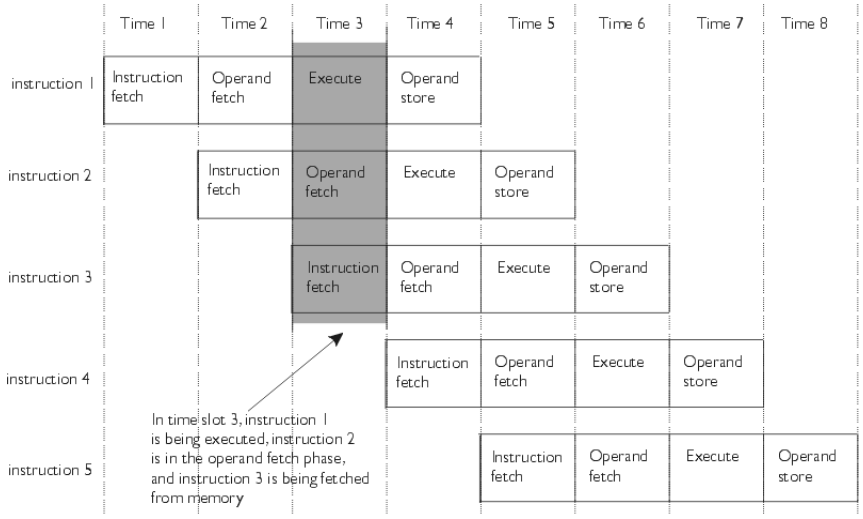
Jim Gray
Turing Award
B.S. Cal 1966
Ph.D. Cal 1969!

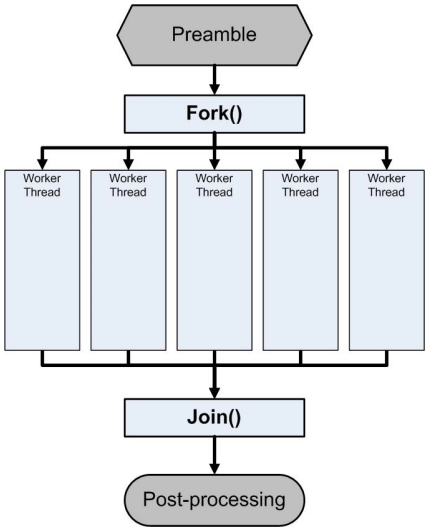


Ideea 3: Localitate - Ierarhia de Memorii



Ideea 4: Paralelism





Ideea 5: Fiabilitate prin Redundanta

- Applies to everything from datacenters to storage to memory to instructors
  - Redundant **datacenters**: can lose 1 datacenter but Internet service stays online
  - Redundant **disks**: can lose 1 disk but not lose data (Redundant Arrays of Independent Disks - RAID)
  - Redundant **memory** bits: can loose 1-2 bits but no data is lost (Error Correcting Code - ECC Memory)





Ce am aflat in aces curs?

- Despre notarea de la ASC
- Sumarul cursului de ASC
- Bibliografia cursului de ASC
- Despre educatia Inginereasca
- De ce Arhitectura Sistemelor de Calcul?
- Idei Importante in Arhitectura Sistemelor de Calcul

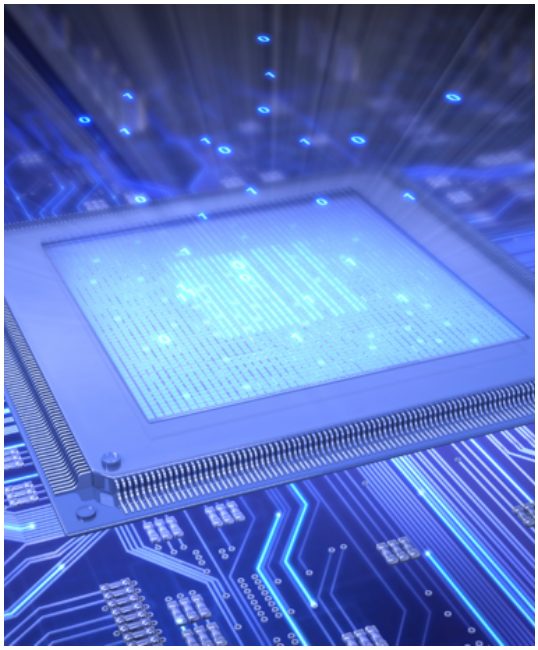


In cursul urmator

- Componentele Sistemelor de Calcul
- Nivelul Structural de Descriere al Sistemelor Numerice – PMS (Processor Memory Switches)
- Exemple de descrieri PMS
- Structuri multiprocesor organizate pe o magistrala comuna (SBC)



Slides & Support Material Acknowledgements



Yale Patt - University of Texas at Austin 2007
CSE 331 - Penn State University 2007
CS61C - University of Berkeley 2016
CS/ECE 3810 - University of Colorado 2017
Ryan J. Leng

Thank you for your attention

emil.slusanschi@cs.pub.ro
cs.pub.ro